Python Port Scanner Tutorial

Contents

Python Port Scanner Tutorial 1
What is Port Scanning? 1
Tutorial 1: A Python Port Scanner 2
Tutorial 2: Rich Port Scanner 5
Tutorial 2: Fast Port Scanner 8
Conclusion 11
Assignment Submission 12

Time required: 60 minutes

Run on both Linux and Windows.

Objective: Write a cross platform Python script that scans ports on a specified IP address.

What is Port Scanning?

Port scanning may be defined as a surveillance technique, which is used in to locate the open ports available on a particular host. Network administrator, penetration tester or a hacker can use this technique. We can configure the port scanner according to our requirements to get maximum information from the target system.

Port scanning is just like a thief who wants to enter a house by checking every door and window to see which ones are open. As discussed earlier, TCP/IP protocol suite, use for communication over internet, is made up of two protocols namely TCP and UDP. Both of the protocols have 0 to 65535 ports. As it always advisable to close unnecessary ports of our system hence essentially, there are more than 65000 doors (ports) to lock. These 65535 ports can be divided into the following three ranges –

- **System or well-known ports:** 0 to 1023
- **User or registered ports:** 1024 to 49151
- **Dynamic or private ports:** > 49151

In this tutorial, you will be able to make your own port scanner in Python using socket library. The basic idea behind this simple port scanner is to try to connect to a specific host (website, server, or any device connected to the Internet/network) through a list of ports, if a successful connection has been established, that means the port is open.

When you loaded a web page, you have made a connection to the website on port 443. This script will try to connect to a host on multiple ports. These kinds of tools are useful for hackers and penetration testers.

WARNING: Never use a tool like this on a host that you don't have permission to test!

Tutorial 1: A Python Port Scanner

This Python script **port_scanner.py** attempts to determine if each port between 0 and 1023 on a specified host is open or closed.

In this section, we will write a simple port scanner.

```
#!/usr/bin/python3
"""
    Filename: port_scanner_simple.py
    Scans each port with a connection attempt
"""

import socket

# Set a default socket connection timeout of .25 seconds
socket.setdefaulttimeout(0.25)
```

Let’s start by importing socket module, which is are part of the Python standard library.

The **socket** module provides us with socket operations, functions for network-related tasks, etc. They are widely used on the Internet, as they are behind any connection to any network. All network communication goes through a socket.

We set a socket connection timeout of 250 milliseconds using **settimeout(0.25)** method. This is the maximum time the script will wait for a connection attempt before considering it unsuccessful.. We are scanning one port at a time, if we don’t set a time out, we could be scanning for a very long time.

```
def is_port_open(host: str, port: int) -> bool:
    """
```

This function checks if the specified port on the given host is open.

Arguments:

host -- The IP address of the host to check

port -- The port number to check

Returns: True if the port is open, False otherwise

```
    # Create an IPV4 TCP socket
    sock = socket.socket(
        socket.AF_INET,     # Address family for IPV4
        socket.SOCK_STREAM  # Socket type for TCP
    )
```

We create a socket object named sock to connnect to an IPV4 IP address with TCP.

```
    # Try to connect to the host using the specified port
    if sock.connect_ex((host, port)):
        # If connection fails, the target port is closed
        # Close the socket object to release OS resources
        # used by opening the socket.
        sock.close()
        return False
```

If the socket object doesn’t make a connection, we return false. The target port is closed. We close the socket object to release the socket operating system resource.

```python
    else:
        # Connection was established, so the target port is open!

        # Send a 'Banner_query' message to the port
        sock.send(b'Banner_query\r\n')

        try:
            # Attempt to receive a response of up to 100 bytes
            results = sock.recv(100)
            # Print the response to the console
            print(f" [+] {results.decode()}", end="\r")
        except:
            # If there is an exception because there wasn't a response
            # from the host, we don't do anything
            pass
        finally:
            # Clean up OS resources used by opening the socket
            sock.close()
        return True
```

If the connection is successful, it means the port is open. It sends a **Banner_query** message and attempts to receive a response. If a response is received (up to 100 bytes), it prints the decoded response as a message. A banner query attempts to get a response from the target, to determine what service is running that port.

```python
# Get the host IP address from the user
host = input(" Enter the host IP: ")

# Iterate over well know ports, from 0 to 1023,
# to check if they are open or closed
for port in range(0, 1024):
    if is_port_open(host, port) ==  True:
        # If the port is open, print a message indicating it is open
        print(f" [+] {host}:{port} is open      ")
    else:
        # If the port is closed, print a message indicating it is closed
        print(f" [-] {host}:{port} is closed    ", end="\r")
```

- The main part of the script prompts the user to input the host's IP address.
- It then iterates through ports from 0 to 1023, checking if each port is open or closed using the is_port_open function.

- Depending on whether the port is open or closed, it prints a corresponding message for each port checked. If open, it prints a message indicating the port is open. If closed, it prints a message indicating the port is closed.

Run the scan.

1. Start and logon to your metasploitable2 VM.
2. Use **ip a** to find out the ip address.
3. Scan the metasploitable ip address from your Kali Linux VM. Make sure each VM is on the same network. You can use a bridged adapter for this exercise.

Example run:

```
Enter the host IP: 192.168.9.118
[+] 220 (vsFTPd 2.3.4)s closed
[+] 192.168.9.118:21 is open
[+] SSH-2.0-OpenSSH_4.7p1 Debian-8ubuntu1
[+] 192.168.9.118:22 is open
[+] 192.168.9.118:23 is open
[+] 220 metasploitable.localdomain ESMTP Postfix (Ubuntu)
[+] 192.168.9.118:25 is open
[+] 192.168.9.118:53 is open
[+] <html><head><title>Metasploitable2 - Linux</title></head><body>
<pre>

[+] 192.168.9.118:80 is open
[+] 192.168.9.118:111 is open
[+] 192.168.9.118:139 is open
[+] 192.168.9.118:445 is open
```

You can see from this example that metasploitable is running OpenSSH on port 22, an SMTP server on port 25, and a web server on port 80.

Tutorial 2: Rich Port Scanner

Rich is a Python library for writing *rich* text (with color and style) to the terminal, and for displaying advanced content such as tables, markdown, and syntax highlighted code.

We are going to use Rich to make our port scanner look more professional

```
# Install the rich library
pip install rich
```

s.connect((host, port)) function tries to connect the socket to a remote address (host, port), it will raise an exception when it fails to connect to that host, that is why we have wrapped that line of code into a try-except block, so whenever an exception is raised, that's an indication for us that the port is actually closed, otherwise it is open.

```python
#!/usr/bin/python3
"""
    Name: port_scanner_simple.py
    Author:
    Created:
    Purpose: Scan each port with a TCP connection attempt
"""

import socket
import time
# Windows: pip install rich
# Linux: pip3 install rich
# import Console for base console printing
from rich.console import Console
# import Panel for title displays
from rich.panel import Panel
# Initialize rich.console
console = Console()
# Set a default connection timeout of .25 seconds
# Otherwise, the scan will take forever
socket.setdefaulttimeout(0.25)


def main():
    # Print program title using rich.panel
    console.print(
        Panel.fit("        Simple Port Scanner        ",
                  style="bold blue",
                  subtitle="By William Loring")
    )

    target_host = input(" Enter the host name/IP to be scanned: ")

    try:
        # Get the IP address of the host
        target_ip = socket.gethostbyname(target_host)
        port_scanner(target_ip)
    except:
        print(f" Unknown host {target_host}")
```

```python
# ----------------------------- PORT SCANNER -----------------------------#
def port_scanner(target_ip: str):
    # Get current time as start time
    start_time = time.time()
    print(f" Starting port scan on host: {target_ip}")

    # Scan from ports 0 - 1023
    for i in range(0, 1024):
        # Print on the same line: ,            end="\r"
        # The extra spaces are to ensure that the previous line is overwritten
        # console.print uses rich.console to make it pretty
        display = f"[green] Scanning port:[/green] [bold green]{i}[/bold green]"
        console.print(display,        end="\r")

        # Create a IPV4 TCP socket
        s = socket.socket(
            socket.AF_INET,     # IPV4
            socket.SOCK_STREAM  # TCP
        )

        # Attempt to make a connection
        # connect_ex returns a 0 for a successful connection
        connection = s.connect_ex((target_ip, i))

        if (connection == 0):
            console.print(f"[bold red] [+] Port {i}: OPEN[/bold red]")
        # Close the socket
        s.close()

    # Calculate and display total scan time
    scan_time = time.time() - start_time
    print(f" Time taken: ({round(scan_time, 2 )})sec")


if __name__ == '__main__':
    main()
```

The above code will scan ports ranging from 1 all the way to 1024, you can change the range to 65535 if you want, but that will take longer to finish.

When you try to run it, you'll immediately notice that the script is quite slow.. This can reduce the accuracy of the reconnaissance, especially when your latency is quite high. We need a better way to accelerate this.

Example run:

To test your port scanner, use a device on your network, or lab.wncc.net

```
Simple Port Scanner
By William Loring
Enter the host name to be scanned: lab.wncc.net
Starting port scan on host: 198.206.239.241
[+] Port 80: OPEN
[+] Port 443: OPEN
Scanning port: 453
```

Tutorial 2: Fast Port Scanner

Let’s take our simple port scanner to a higher level. In this section, we'll write a threaded port scanner that is able to scan 200 or more ports simultaneously.

The below code is the same function we saw previously, which is responsible for scanning a single port. Since we're using threads, we need to use a lock so only one thread can print at a time, otherwise, the output will be messed up and we won't read anything:

We used **Queue()** class from the built-in queue module that will help us with consuming ports, the two below functions are for producing and filling up the queue with port numbers and using threads to consume them:

We get port numbers from the queue and scan it, and then add it to the done tasks, whereas `main()` function is responsible for filling up the queue with the port numbers and spawning `N_THREADS` threads to consume them.

Note the q.get() will block until a single item is available in the queue. q.put() puts a single item into the queue and q.join() waits for all daemon threads to finish (clearing the queue).

```python
#!/usr/bin/python3
"""
    Filename: port_scanner_threaded_queue.py
    Scans each port with a connection attempt
"""

import queue
import socket
import time
import threading
import rich
# Windows: pip install rich
# Linux: pip3 install rich
# import Console for base console printing
from rich.console import Console
# import Panel for title displays
from rich.panel import Panel
# Initialize rich.console
console = Console()


class PortScanner:
    def __init__(self):
        # Print program title using rich.panel
        console.print(
            Panel.fit("        Fast Port Scanner        ",
                      style="bold blue",
                      subtitle="By William Loring")
        )
        # Set a default connection timeout of .5 seconds
        socket.setdefaulttimeout(0.5)
        # Define a thread lock to keep
        # print statements from writing over themselves
        self.thread_lock = threading.Lock()
        self.ports = queue.Queue()
        self.num_threads = 200
        target_host = input(" Enter the host to be scanned: ")
        # Get the IP address of the host
        self.target_ip = socket.gethostbyname(target_host)
        print(f" Starting scan on host: {self.target_ip}")
        self.start_time = time.time()
        self.count = 0
        # Scan the first 5000 ports maximum ports 65535
        port_range = 5001
        for port in range(1, port_range):
            self.ports.put(port)
```

```python
# ---------------------------- SCAN PORT -------------------------------#
    def scan_port(self, port):
        try:
            s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            result = s.connect_ex((self.target_ip, port))
            if result == 0:
                # thread_lock prevents the threads from
                # running into each other
                with self.thread_lock:
                    self.count += 1
                    console.print(f"[green] [+] Port {port}: OPEN[/green]")
            s.close()
        except:
            pass

# ---------------------------- WORKER ----------------------------------#
    def worker(self):
        while not self.ports.empty():
            port = self.ports.get()
            self.scan_port(port)

# ---------------------------- RUN -------------------------------------#
    def run(self):
        threads = []
        for i in range(self.num_threads):
            t = threading.Thread(target=self.worker)
            threads.append(t)
            t.start()

        for t in threads:
            t.join()
        scan_time = time.time() - self.start_time
        print(f" {self.count} ports open.")
        print(f" Time taken: ({round(scan_time, 2)})sec")


scanner = PortScanner()
scanner.run()
```

Example run:

Example run against a Windows 2022 Server.

Scanning my home server.

```
Fast Port Scanner
By William Loring
Enter the host to be scanned: 01lenovo
Starting scan on host: 192.168.9.10
[+] Port 53: OPEN
[+] Port 88: OPEN
[+] Port 135: OPEN
[+] Port 139: OPEN
[+] Port 389: OPEN
[+] Port 445: OPEN
[+] Port 464: OPEN
[+] Port 593: OPEN
[+] Port 636: OPEN
[+] Port 3268: OPEN
[+] Port 3269: OPEN
[+] Port 3389: OPEN
Time taken: (12.9)sec
```

```
Fast Port Scanner
By William Loring
Enter the host to be scanned: lab.wncc.net
Starting scan on host: 198.206.239.241
[+] Port 80: OPEN
[+] Port 443: OPEN
[+] Port 541: OPEN
[+] Port 990: OPEN
Time taken: (12.89)sec
```

Conclusion

Awesome! The new program finished scanning 5000 ports in less than 15 seconds! You can use the default range (1 to 65535) and it will take only a bit longer to finish.

If you see your scanner is freezing on a single port, that's a sign you need to decrease your number of threads, if the server you're probing has a high ping, you should reduce N_THREADS to 100, 50, or even lower, try to experiment with this parameter.

Port scanning proves to be useful in many cases, an authorized penetration tester can use this tool to see which ports are open and revealing the presence of potential security devices such as firewalls, as well as testing the network security and the strength of a device.

It is also a popular reconnaissance tool for hackers that are seeking weak points to gain access to the target machine.

Most penetration testers use **nmap** to scan ports, as it does not just provide port scanning, but shows services and operating systems that are running, and much more advanced techniques.

Assignment Submission

1. Attach the program files.
2. Attach screenshots showing the successful operation of the program.
3. Submit in Blackboard.